

Reproducible Research in Practice: Collaborative Workflows for Data Analysis

Zhongyan Chen ¹ Shunqi Zhang ² Shaun Scholes ²

¹Imperial College London

²University College London

28/07/2026

Workshop Instructors



Shunqi Zhang

Github: @SZhang97

Linkdlen: @ShunqiZhang

Workshop Instructors



Zhongyan Chen

Github: @ZhongyanChen

Linkdlen: @zhongyanchen



Shaun Scholes

Github: @shauns11

Linkdlen: @shaunscholes

Meet your TA!

What if I am in over my head?

How are you feeling?
ONE word is enough!

Meet your neighbours!
Please add folks on LinkedIn if you are comfortable to do so! It would be helpful to improve your **visibility**!

Agenda

1. Motivation and Introduction
2. Pre-workshop Clinic
3. Coffee Break
4. Git: Single Player Mode
5. Lunch Break
6. Project Folder Structure
7. Git: Multiplayer Mode
8. Closing Note

Yes, materials are available.

All will be revealed and downloaded.

If we have it, you can have it.

Motivation and Introduction

A set of pics to self-describe your code management and coding practice

Why do we do a workshop on reproducibility?

Software Checklist

- Git + Git GUI/GitHub
- Development Environment
 - Python** Users: Python + PyCharm
 - R** Users: R + RStudio
 - Stata** Users: Stata

➤ Check if **Python** is successfully installed:

❖ **Windows users:**

Type this command in your cmd:

```
python --version
```

❖ **Mac users:**

Type in your terminal:

```
python --version
```

or

```
python3 --version
```

- Check if **Git** is successfully installed:
Type this command in your cmd/terminal:

```
git -v
```

Have you experienced...

 Report_v1.1_Final_Version_Before_Review.docx

Or...

 updated_gee.RData

Do you...

- forget which file is the most up-to-date version?
- worry about the authorship of your code?
- share code by email (multiple times and versions)?
- Git can solve these!

Introduction to Version Control

- What is version control?
An important practice in software development for tracking and managing changes made to **code** and **other files**.
- Today we focus on **source code** (i.e., the .py/.rmd/.do files).

Introduction to Version Control

How can Version Control help me to Collaborate Efficiently?

- ❖ Serves as a **safety net** to protect the source code from irreparable harm.
- ❖ Records not just the change, but also the **person** who made it. (authorship!)
- ❖ **Multiple developers** can work on the same project simultaneously without overwriting each other's changes.



➤ Software Involved and their roles

❖ **PyCharm/RStudio/VS Code**

- ✓ Integrated Development Environment (IDE)
- ✓ Provides a modern graphical user interface (GUI) for Git
- ✓ Enhances code formatting and version control integration

❖ **Git**

- ✓ The most popular Version Control System (VCS)
- ✓ Command line or a basic GUI
- ✓ Enables efficient tracking and collaboration on code changes

❖ **Stata:**

- **NO** VCS integration

A Mini Project Practicing Git

- A miniature data science project done in Python/R/Stata
- Aim: to practice Git, not how to code
- Needs to know basic statistics
- Code provided

Create Your First Repository

- Create an empty repository on GitHub and add the README.
- Copy it to the local machine (aka **clone**)

What is a Repository?

- It's an invisible `.git/` folder inside the project root directory.
- It contains all versions tracked by Git.
- More information [here!](#)

Local Repository vs. Remote Repository

- We now have two repositories: one on our machines (local) and another on GitHub's server (remote).
- Functionally, these two repositories are the same.
- We treat the remote repository as the central repository, as it is where our code is published.

Make your First Commit

- Copy/paste the data file and source code file.
- The code is a `.py/.rmd/.do` file that prints data to the console.
- Commit your `.py/.rmd/.do` file.
- A commit is a snapshot of your repo at a specific point in time.

New Feature: Query Specific Data and Write to a File

- Modify the `.py/.rmd/.do` file so it can find all data about **setosa** and write it to a csv file.
- Before commit: what about the *setosa.csv* file?

The .gitignore File

- A *.gitignore* file tells Git which files and folders that you **DO NOT** wish it to track in your repository.
- **Stata users** can manually unstage the file(s) and do not need to create a *.gitignore* file.

What files/folders may we not want Git to track?

Examples:

- Data files (e.g., *Iris.csv*, *setosa.csv*)
- Files that are too large.
- Other irrelevant files/folders (e.g., build outputs, logs, editor files...)

New Feature: Query Specific Data and Write to a File

- Modify the `.py/.rmd/.do` file so it can find all data about **setosa** and write it to a csv file.
- Before commit: what about the `setosa.csv` file?
- Commit the modified `.py/.rmd/.do` and `.gitignore` files.

Push the Change to Remote Repository

- Update remote repository with the changes made in the local repository (aka **push**)
- Refresh then check GitHub's webpage
- Be careful before pushing

Making a commit: Tips

The quality of each version should be good for better version control.
Here are tips about some good practices when making a commit

- **Commit Changes Atomically:** making the smallest possible changes in a commit

Making a commit: Tips

The quality of each version should be good for better version control.
Here are tips about some good practices when making a commit

- **Commit Changes Atomically:** making the smallest possible changes in a commit
- **Commit with a Single Purpose:** Each commit should have a single purpose. For example, fixing a bug or adding a new feature. If a single change makes multiple independent changes to your project, it can become difficult to read and review.

Making a commit: Tips

The quality of each version should be good for better version control. Here are tips about some good practices when making a commit

- **Commit Changes Atomically:** making the smallest possible changes in a commit
- **Commit with a Single Purpose:** Each commit should have a single purpose. For example, fixing a bug or adding a new feature. If a single change makes multiple independent changes to your project, it can become difficult to read and review.
- **Write Meaningful and Concise Commit Messages:** aim to explain what this commit is about and why these changes are made, which also depends on how many changes a commit contains.

Making a commit: Tips

The quality of each version should be good for better version control. Here are tips about some good practices when making a commit

- **Commit Changes Atomically:** making the smallest possible changes in a commit
- **Commit with a Single Purpose:** Each commit should have a single purpose. For example, fixing a bug or adding a new feature. If a single change makes multiple independent changes to your project, it can become difficult to read and review.
- **Write Meaningful and Concise Commit Messages:** aim to explain what this commit is about and why these changes are made, which also depends on how many changes a commit contains.
- **Use previews to review changes before commit:** Review your change before commit. Once a commit is made, it cannot be undone!
- In an event of a fire alarm: git commit, git push, evacuate!

Making a Commit: Workflow

Let's summarise the steps for making a commit to push the changes to the remote repository.

- Do some coding
- Prepare to commit: code review
- Commit changes with a meaningful message
- Push the changes to the remote repository

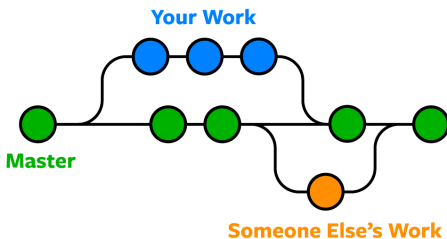
View diff on GitHub and PyCharm/RStudio/Git GUI

With the two commits we just made, we can:

- Use **checkout** to examine a specific commit
- View the change made between two commits (aka **diff**)
- If you try to commit based on an old commit, Git will reject the commit as the project is in a detached HEAD state.

Creating Your First Branch

- Conceptually, a branch is a line of commits.
- Technically, a Git branch is just a pointer.
- Git creates a default branch when the repository is created (named "main" or "master" by default).
- It's useful to have another branch when we need to work on some code without affecting other branches (mostly the main branch).
- Now, let's create a new branch called "revert-demo"



Revert a Commit

- Revert a commit in the new branch
- Reverting a commit is **NOT** recommended
- Once pushed, the commits cannot be undone!
- Preview the diff before making the commit or pushing.



More on Branching

- Just now, we may have caused some chaos resulting in a confusing commit history, but it does not affect the main branch or the remote repository at all.
- Experimental development is advised to be carried out in a **separate** branch.

Delete a Branch

- Delete the "revert-demo" branch
- A branch can be deleted on both local and remote repos

Fix the Bug in another Branch

- Create another branch called "fix-write-data-bug"
- Fix the bug
- Commit the change

Merge a Branch

- Now that the bug has been fixed, the change can be added to the main branch.
- N.B.: Make sure that the order in which you merge the branches is correct: We want merge the development branch into the main branch.

Git on Single Player Mode: Workflow

If you have a development task (add new features/fix a bug):

- 1. Create a feature branch
- 2. Make commits
 - 2.1 Do some coding
 - 2.2 Prepare to commit: code review
 - 2.3 Commit to the feature branch
 - 2.4 Push the feature branch
- 3. Merge into the main branch
- 4. Push the main branch

Lunch Break Time

➤ Let's have the lunch break!

What is your style of file management?

Be Organised!

- Self-explanatory!
Enable "**someone**" to look at your files and understand in detail what you did and why!
That **someone** includes your future self
(or your future AI assistant because it needs to understand your folder structure)!
- So you need to establish a folder structure.
- Any structure is better than none!
- But also don't fret over past mistakes raise the bar for **NEW** folder structure.

Organisation Practices: Establish a Folder Structure

- Plan out folder hierarchy and file naming in the beginning.
- Consider how you or others will search and access files later.
- Discuss with your project team to set up a way for easy adoption.
e.g., A README.

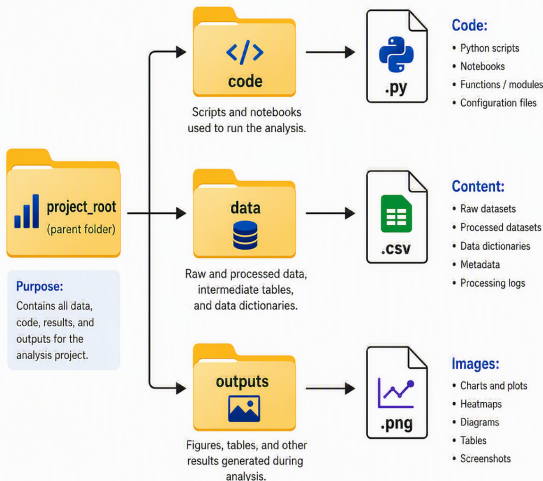
Organisation Practices: Organise Folder Hierarchy

- One project, one folder.
- Avoid overlapping categories.
- Consider **sub-folders** when a folder has too many files.
- Don't let your structure be too complicated.
- Never overwrite your MASTER dataset. Protect it! Always run the function on the RAW data.

Organisation Practices: Organise Folder Hierarchy

DATA ANALYSIS PROJECT STRUCTURE

A typical organization of files and outputs in a data analysis project



Organisation Practices: Create Folder Attribute

- Identify ways to classify files to categories and attributes.
- Consider the possible number of files and what aspects are the most essential.

Organisation Practices: Create Documentation Files

A good README is:

- updated.
- concise.
- crystal clear.

Using Git: Multiplayer Mode

- Collaboration practice
- Task: To add two features, done by two people.

How does this Activity Work?

- We pair two people in a group to do the activity.
- Each person will develop a feature independently.
- You can discuss with your teammate who takes which feature.
- Code provided.

The Task Features

- Player 1. Print the mean petal length on the console.
- Player 2. Print the species frequency data on the console.

Create a Local Branch for Yourself

- Player 1: Create a local branch called "**mean-petal-length**"
- Player 2: Create another local branch called "**species-frequency-data**"
- This allows everyone to work in parallel without affecting each other or the main branch.

Commit Your Change

- Follow the instructor's demonstration.
- Commit your change to the local feature branch.

Push Your Change to the Remote Repository

- Player 1 & 2: Push the local feature branch to the remote repository.

Merge branches: Multiplayer mode

- Now let's try to merge the changes made by two people into the main branch.

Feature 2 Developers...

- Merge your local feature branch into the local main branch
- Push your local main branch to the remote repository.

Feature 1 Developers...

- Wait before you merge!
- What would be different if I attempt to merge and push?
- Short answer: Messing up the commit history

Why is there the difference?

- In this example, feature 2 developers pushed their work faster than ours.
- This makes our local repository's version fall behind.
- To maintain a clean commit history:
- Before merging into the local branch, we need to update our local repository first.

Update your Local Repository

Feature 1 Developers...

- Update "origin/main", which effectively **ONLY** updates the git graph (aka **Fetch**).
- **Git GUI** Users: Update the files in the local repository by **merging** "main" into "origin/main"
- **PyCharm/RStudio** Users: **pull**

More on **Pull**

- Effectively, **pull** = **fetch** + **merge** "main" into "origin/main".
- **Fetch** updates the git graph without changing files in the local repository.
- **Pull** will change the files in the local repository.
- Since **fetch** is always safe, it is recommended to **fetch** first, then **pull**.

Feature 1 Developers...

- Fetch
- Pull
- Merge your local feature branch into the local main branch
- Resolve conflicts
- Push your local main branch to the remote repository.

Git on Multiplayer Mode: Workflow

- 1. Create a feature branch
- 2. Make commits
 - 2.1 Do some coding
 - 2.2 Prepare to commit: code review
 - 2.3 Commit to the feature branch
 - 2.4 Push to the remote feature branch
- 3. **Fetch** remote tracking branches
- 4. **Pull** remote repository's changes
- 5. Resolve conflicts and merge into the local main branch.
- 6. Push the changes to the remote main branch

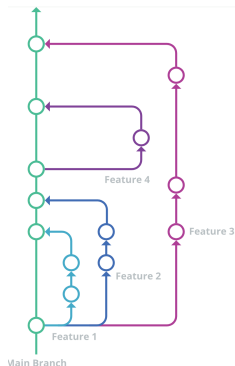
Coding as a Team: Tips & Tricks

Objective: Maintaining a clean commit history

- **Keep the main branch ready-to-go:** The code in the main branch should always be ready to be used by you/your team.
- **Create short-lived feature branches:** A branch should contain a few commits dedicated to one task (e.g., adding a feature or fixing a bug).
- **Merge frequently:** Merge smaller changes to the code into the main branch to reduce the workload for code review, resolving conflicts, etc.
- **Always update the local repository before merging.**

Branching Strategies

- What we've done so far is also known as the "GitHub" workflow.



Branching Strategies

- What we've done so far is also known as the "GitHub Workflow".
- There are other branching strategies: Git Workflow, GitLab Flow, etc.
- Each strategy has its pros & cons.

- Publish your code
- Project visibility
- Add collaborators to your project
- **Fork** other people's projects.
N.B., Forks are often used to iterate on ideas or changes before they are proposed back to the original repository, such as in open source projects or when a user does not have write access to the upstream repository.

What are Pull Requests?

- A pull requests is a proposal to merge code changes into a project.
- It is GitHub's key collaboration feature.
- It allows discussions and reviews of proposed code changes before merging them.

Pull Requests: Basic Usage

- Only need to watch instructor's demonstration.
- Demonstration: review each team member's code before merging.

Pull Requests: Advanced Usage

- Do you want to contribute to your favorite R package?
- Do you want to promote your research tool to the wider community/industries?
- Watch this demonstration: contribute to other people's projects.

Further Reading Material

- Git Pro Book
- GitKraken Tutorial
- Got some specific questions? Try Stack Overflow!
- It's 2026: ask AI!

➤ Any Questions?

Acknowledgment

- We thank the UCL OD Academic and Researcher Experience Team and the UCL Grand Challenges Team for providing funding of this workshop.

The End

- Thank you for joining the workshop!
- Please complete the workshop survey!

The End

